
NECST: VIMCO vs. Gumbel–Softmax Optimization

Aditya Mittal

Department of Computer Science
University of California, Irvine

Nick Du

Department of Computer Science
University of California, Irvine

ZhaoBin Li

Department of Cognitive Science
University of California, Irvine

Abstract

Our project reimplements the NECST framework, a compression–transmission mechanism that is resilient to channel noise. Classical information theory shows that source and channel coding can be optimized separately in the asymptotic limit of infinitely long messages. However, under finite bit-length constraints, jointly learning compression and error correction becomes more effective. NECST uses a discrete variational autoencoder that injects noise into latent representations during training. Since the latent codes are discrete and non-differentiable, the original work uses VIMCO for low-variance gradient estimation. We reimplement VIMCO and extend the framework by including Gumbel–Softmax as an alternative optimization approach, comparing both methods on reconstruction quality, latent robustness, and downstream classification using MNIST and Omniglot. The code is here: <https://github.com/adityamittal03/necst-torch>.

1 Introduction

This report presents a reimplementation of the model introduced in *Neural Joint Source-Channel Coding* (NECST) [1]. Classical information theory shows that source and channel coding can be optimized independently with infinite bits, but under finite bit-length constraints this separation becomes suboptimal. NECST addresses this by jointly learning compression and error correction using a discrete variational autoencoder.

The original paper discusses communication as a variational inference problem: (i) the encoder maps images to discrete latent bits parameterized by Bernoulli variables, (ii) the channel adds noise to simulate transmission errors, and (iii) the decoder reconstructs the input through amortized inference. The training process maximizes the variational lower bound on the mutual information between the input and noisy latent representation using VIMCO [5] to ensure the learned codes are compact and robust. This joint formulation creates compressed representations and error-correcting codes, and amortized decoding reduces the computational cost during inference.

In this project, we reimplement the NECST model and compare our results with the original paper. Our contributions are:

- A PyTorch reimplementation of the NECST model.
- Included Gumbel–Softmax as an alternative to VIMCO, with an empirical comparison of both optimization methods.
- An evaluation of the learned latent codes using reconstruction error, robustness under channel noise, and downstream classification accuracy.

2 Optimization

The main challenge in training the model end-to-end is that sampling from discrete latent variables is non-differentiable. Since the discrete bits are generated through a non-differentiable sampling step, gradients cannot flow backward and standard backpropagation fails. This creates an optimization bottleneck: the encoder must learn a distribution over discrete codes, but the gradients cannot propagate through the sampling step.

To address this, we implement two optimization methods for training with discrete latent variables: (i) VIMCO [5], the low-variance importance-weighted gradient estimator used in the original paper, and (ii) Gumbel–Softmax [2], a differentiable relaxation that enables gradient flow. We describe both approaches below.¹

2.1 VIMCO Loss for Discrete Latents

VIMCO uses an importance-weighted variational objective over k stochastic bit samples. For an input x , encoder $q_\phi(z | x)$, decoder $p_\theta(x | z)$, and samples $\{z_i\}_{i=1}^k \sim q_\phi(\cdot | x)$, the bound is

$$\mathcal{L}(x) = \mathbb{E}_{z_{1:k} \sim q_\phi} \left[\log \frac{1}{k} \sum_{i=1}^k w_i \right], \quad w_i = \frac{p_\theta(x | z_i) p(z_i)}{q_\phi(z_i | x)}.$$

While the $\mathcal{L}(x)$ objective tightens with larger k , it introduces high-variance gradients for ϕ because the latent bits are discrete. VIMCO (Variational Inference for Monte Carlo Objectives) mitigates this by constructing per-sample control variates from the remaining $k - 1$ samples, producing a leave-one-out learning signal for each z_i .

In practice, VIMCO splits the gradient into separate “ θ ” (decoder) and “ ϕ ” terms (encoder). The decoder uses the standard weighted likelihood gradient. The encoder uses a variance-reduced estimator: for each sample i , it replaces $\log \frac{1}{k} \sum_j w_j$ with a baseline that swaps out w_i for an average over the other $k - 1$ weights. This keeps the estimator unbiased while weakening the coupling between z_i and its own gradient.

2.2 Gumbel–Softmax with Straight-Through (Hard) Estimator

As a second differentiable alternative, we relax each Bernoulli bit using the binary Gumbel–Softmax (Gumbel–Sigmoid). Given logits α , we draw noise $g = \log u - \log(1 - u)$ with $u \sim \text{Uniform}(0, 1)$ and form

$$y = \sigma \left(\frac{\alpha + g}{\tau} \right),$$

where τ is the temperature. As $\tau \rightarrow 0$, y approaches a binary value; for larger τ , the relaxation becomes smoother and yields smaller variance gradients. The relaxed y propagates through the decoder $p_\theta(x | y)$ to allow backpropagation while approximating discrete sampling.

To match test-time binarization during training, we use the straight-through (“hard”) variant: the forward pass thresholds y to $y_{\text{hard}} = \mathbf{1}[y > 0.5]$, while the backward pass copies gradients from the continuous y (i.e., stop-gradient on the rounding). Training then minimizes a reconstruction loss (binary cross-entropy on logits) plus a KL term encouraging a Bernoulli(0.5) prior over the bits. This setup achieves discrete behavior at inference and maintains stable optimization during training.

3 Method

We replicate the key experimental setups from the original paper and compare VIMCO and Gumbel–Softmax optimization. We first describe the datasets and model architecture, then evaluate both methods on: (1) reconstruction error under varying channel noise levels, (2) latent space interpolation and Markov chain visualization, and (3) downstream classification accuracy using the learned binary codes.

¹The original paper claims that VIMCO empirically outperforms Gumbel–Softmax. We implement and compare both methods.

3.1 Datasets

We use the same datasets as the original paper: binarized MNIST [4] and Omniglot [3]. MNIST contains handwritten digits 0-9, and Omniglot is a more challenging dataset of handwritten characters across 50 writing systems. Following the original paper, we binarize both datasets by rounding pixel values to $\{0, 1\}$. For MNIST, we use train/validation/test splits of 50K/10K/10K. For Omniglot, we use splits of 23,845/500/8,070. Example images are shown in Figure 1. The original paper also included RGB datasets such as CIFAR-10, SVHN, and CelebA, but we omitted those in our analysis.

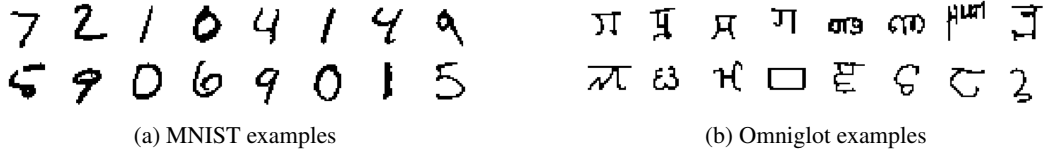


Figure 1: Example images from MNIST and Omniglot.

3.2 Model Architecture

Following the original paper, we use a simple MLP-based encoder-decoder architecture. The encoder is a 1-hidden-layer MLP with 500 hidden units and ReLU activations, mapping input images to latent bit logits. The decoder is a 2-layer MLP with 500 hidden units per layer also using ReLU activations. The final decoder layer applies a sigmoid to parameterize the Bernoulli likelihood ($p_\theta(x | z)$). For MNIST, we generate 100 latent bits; for Omniglot, we use 200 latent bits. We change model optimization to see if we can replicate results from the paper and understand how the optimization results differ.

3.3 Training Details

We train using the Adam optimizer with a learning rate of 0.001, a batch size of 100, and an L2 regularization penalty of 0.001 on the encoder weights. For MNIST, we train for 200 epochs; for Omniglot, we train for 500 epochs. All hyperparameters match those in the original paper. We add noise using a BSC channel with noise level 0.1, which independently flips each latent bit with probability $\epsilon = 0.1$ to simulate a noisy communication channel.

4 Results

This section shows our experiment results. Plots for Omniglot are provided in the appendix.

4.1 Reconstruction Error

The reconstruction errors for Omniglot and MNIST are shown in Table 1. We evaluate reconstructions across noise levels 0.1, 0.2, 0.3, 0.4, and 0.5. Overall, VIMCO consistently achieves lower reconstruction error than Gumbel across all noise levels. For MNIST, the gap becomes large as noise increases (0.292 vs. 0.512 at $\epsilon = 0.4$). This is also true in Omniglot: VIMCO error more stable as noise increases, while Gumbel degrades quickly (0.225 vs. 0.569 at $\epsilon = 0.4$).

Binary MNIST	0.0	0.1	0.2	0.3	0.4
100-bit NECST-VIMCO	0.085	0.101	0.135	0.198	0.292
100-bit NECST-GUMBEL	0.076	0.179	0.292	0.403	0.512
Binary Omniglot	0.0	0.1	0.2	0.3	0.4
200-bit NECST-VIMCO	0.153	0.157	0.166	0.186	0.225
200-bit NECST-GUMBEL	0.162	0.171	0.232	0.356	0.569

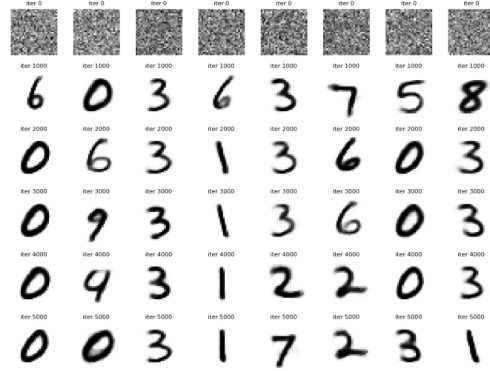
Table 1: Reconstruction error comparison between NECST-VIMCO and NECST-GUMBEL.

4.2 MNIST Plots

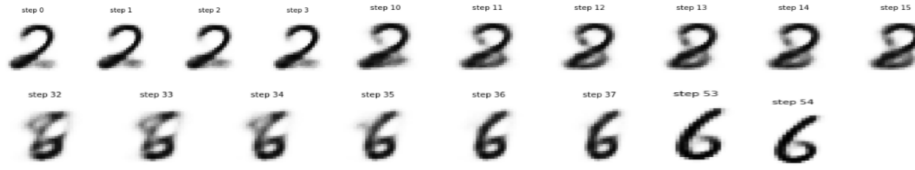
Figure 2 shows MNIST results using VIMCO. The reconstruction plot shows that the decoder recovers digits reliably, with noise causing gradual degradation as ϵ increases. The Markov chain samples show that resampling latent bits generates different numbers from random noise, indicating that the learned codes span a diverse generative space. Lastly, the latent interpolation between digits 2 and 6 shows a smooth transition: the interpolations between their code vectors generate meaningful intermediate images (“8”). This suggests that the learned binary latent space captures some of the underlying structure in the images.



(a) MNIST test reconstructions. Row 0 is original data. Rows 1–5 show reconstructions under noise levels $\epsilon \in \{0.0, 0.1, 0.2, 0.3, 0.4\}$.



(b) Diverse MNIST Markov chain samples generated from random noise using learned latent bits.



(c) Latent interpolation between digits 2 and 6. Leads to an intermediate 8.

Figure 2: MNIST reconstructions, Markov chain visualizations, and latent space interpolation.

4.3 Downstream error

Table 2 reports downstream classification accuracy on MNIST binary latent features using five standard classifiers trained on the learned codes. Across all noise levels, VIMCO consistently produces more informative representations than Gumbel with higher accuracy for every classifier. The gap widens as noise increases, and all models show the same trend: VIMCO retains far more discriminative structure under noise, while Gumbel degrades rapidly. These results align with the reconstruction experiments and confirm using VIMCO loss learns more robust latent codes for downstream tasks.

5 Discussion

Our reimplementation matches the results reported in NECST: discrete latent codes trained under channel noise remain robust and reconstructive, and VIMCO consistently produces stronger representations than the Gumbel–Softmax relaxation. Though the exact accuracy numbers are slightly different from the original paper, both the reconstruction and downstream results follow the same trends, with the performance gap widening as test channel noise increases.

The experiments also highlight the limits of the simple MLP architecture and fixed bit budgets. While our results broadly align with the original work, pushing performance further likely requires

MNIST (Downstream Accuracy)	0.0	0.1	0.2	0.3
100-bit NECST-VIMCO				
Logistic Regression	0.8925	0.8119	0.6887	0.5105
KNN	0.9545	0.9165	0.6846	0.3696
Random Forest	0.9557	0.8833	0.7411	0.4540
SVM	0.8845	0.8064	0.7143	0.5053
XGBoost	0.9631	0.9170	0.7759	0.5307
100-bit NECST-GUMBEL				
Logistic Regression	0.8101	0.6308	0.4550	0.3133
KNN	0.8166	0.5127	0.2711	0.1549
Random Forest	0.8977	0.6740	0.4468	0.2695
SVM	0.8019	0.6277	0.4510	0.3097
XGBoost	0.9274	0.7378	0.5021	0.3091

Table 2: Downstream classification accuracy for MNIST across test noise levels using multiple classifiers on latent codes from NECST-VIMCO and NECST-GUMBEL.

larger, more complex models and adaptive code lengths. Another natural next step is extending the framework to the CNN encoder–decoder used for RGB datasets in the original paper to see how these optimization methods scale to more complex data. It also useful to explore a unified encoder–decoder design that generalizes across datasets instead of maintaining separate models for each one.

5.1 Challenges Faced

We faced several practical challenges during the reimplementation. The original TensorFlow code was outdated and required many changes to fix imports and APIs. Implementing the VIMCO and Gumbel losses in PyTorch also required understanding the reference papers and the TensorFlow code before adapting the methods to our framework. These issues were resolved together and the final model reproduces the results from the original work.

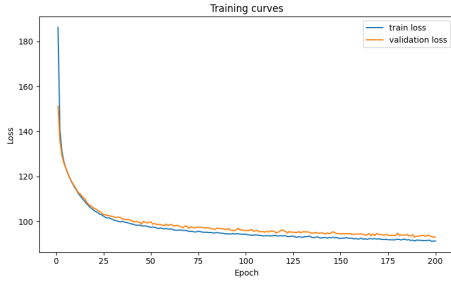
Contributions

The three group members contributed equally: Aditya handled the initial model setup and the PyTorch reimplementation; ZhaoBin prepared and wrote the report; Nick ran the experiments to generate results and produced the dataset plots.

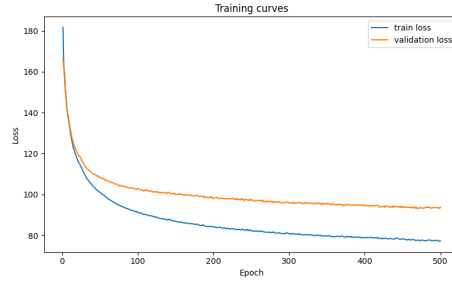
References

- [1] Kristy Choi, Kedar Tatwawadi, Aditya Grover, Tsachy Weissman, and Stefano Ermon. Neural joint source-channel coding. In *International Conference on Machine Learning*, pages 1182–1192. PMLR, 2019.
- [2] Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel-softmax. *arXiv preprint arXiv:1611.01144*, 2016.
- [3] Brenden M Lake, Ruslan Salakhutdinov, and Joshua B Tenenbaum. Human-level concept learning through probabilistic program induction. *Science*, 350(6266):1332–1338, 2015.
- [4] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 2002.
- [5] Andriy Mnih and Danilo Rezende. Variational inference for monte carlo objectives. In *International Conference on Machine Learning*, pages 2188–2196. PMLR, 2016.

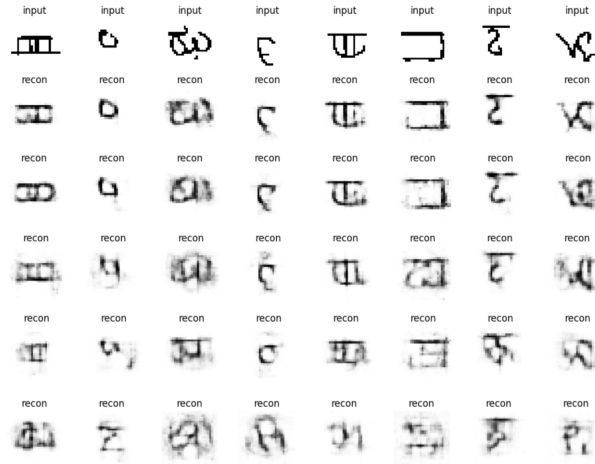
6 Appendix



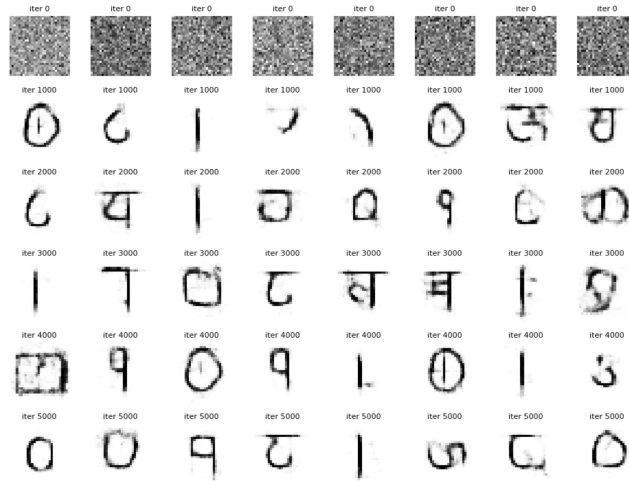
(a) Train/Val loss on MNIST using VIMCO (200 epochs).



(b) Train/Val loss on Omniglot using VIMCO (500 epochs).



(c) Omniglot test reconstructions across noise levels $\epsilon \in \{0.0, 0.1, 0.2, 0.3, 0.4\}$.



(d) Diverse Omniglot Markov chain samples generated from random noise.

Figure 3: Appendix results for VIMCO on MNIST and Omniglot. Top: training and validation loss curves. Bottom: Omniglot reconstructions and Markov chain samples, which are more distorted and noisier than MNIST.